

FixIt — System Architecture & Working Model

Enterprise Distributed Full-Stack Service Request & Repair Network

Version: 1.0.0 | Author: Aman Singh | Status: Production Active | Environment: Multi-Cloud / Kubernetes

Platform Summary: FixIt is an enterprise-scale on-demand repair and service ecosystem connecting Customers, Certified Technicians, and Super Admins. The system features a 5-step diagnostic request wizard, smart auto-matching, real-time bidirectional Socket.IO chat, Razorpay digital payment processing, RabbitMQ task queues with DLQ, Apache Kafka event streaming (KRaft), n8n workflow automation, and production Kubernetes orchestration.

1. System Architecture & Technology Stack

FixIt is architected around a decoupled, micro-service ready layered paradigm. The frontend is a responsive React 18 Single Page Application served via Nginx or Vercel Edge. The backend REST and WebSocket servers are powered by Node.js 20+ Express implementing a Controller-Service-Repository pattern backed by Neon PostgreSQL, Upstash Redis, RabbitMQ task queues, Apache Kafka streaming, and n8n automations.

Layer	Technology	Implementation & Architectural Role
Frontend SPA	React 18, Vite 5, TailwindCSS	Client portal with responsive design tokens, dark/light theme engine, live Socket.IO chat, and 5-step wizard.
REST API Server	Node.js 20+, Express.js	Modular Controller-Service-Repository architecture with Zod schema validation and RBAC guards.
Relational DB	Neon PostgreSQL (Serverless)	ACID-compliant storage, connection pooling, automated Prisma migrations, and auto-seeding.
Task Queue & DLQ	RabbitMQ 3 (Management)	Asynchronous worker queues (emails, notifications, webhooks) with Dead Letter Exchange (fixit.dlx).
Event Streaming	Apache Kafka 3.7 (KRaft)	Distributed log for request lifecycle, audit telemetry, and real-time streaming without Zookeeper.
Workflow Engine	n8n Low-Code Workflows	Event-driven webhook automations: Onboarding alerts, emergency escalation, invoice & CRM sync.
Payment Gateway	Razorpay SDK (Test/Live)	Server-side order generation, HMAC-SHA256 signature verification, and async webhook handlers.
Real-Time Engine	Socket.IO + JWT	Multiplexed WebSocket channels with room routing (user, role, and chat:requestId).
Cloud / DevOps	Docker, K8s, GitHub Actions	Multi-stage non-root containers, Horizontal Pod Autoscaling (HPA 2-10 pods), and CI/CD pipelines.

2. Database Schema & Data Models

The relational schema is managed via Prisma ORM and contains 13 models mapped with rigorous foreign key constraints and performance indexes:

Model	Primary Fields	Relationships & Constraints
User	id, email, passwordHash, role (CUSTOMER, TECHNICIAN, ADMIN), isActive, emailVerified	1-to-1 with TechnicianProfile; 1-to-Many with ServiceRequests, Notifications, Messages, Tokens.
TechnicianProfile	id, userId, bio, experienceYears, verificationStatus, availability, averageRating, totalEarnings	Belongs to User; 1-to-Many with TechnicianService, ServiceArea, and RequestAssignment.
ServiceRequest	id, customerId, serviceId, title, description, status (PENDING..COMPLETED..CANCELLED), priority	Belongs to User & Service; 1-to-1 with Location, Review, Payment; 1-to-Many Images, Messages.
ServiceLocation	id, requestId, address, city, state, zipCode, latitude, longitude	1-to-1 unique mapping to ServiceRequest.
RequestImage	id, requestId, imageUrl, publicId, caption	1-to-Many attachments per ServiceRequest.
RequestAssignment	id, requestId, technicianId, status (PENDING, ACCEPTED, REJECTED), assignedAt	Join model linking ServiceRequest to matched TechnicianProfiles.
Message	id, requestId, senderId, receiverId, content, isRead, createdAt	Stores bidirectional chat history linked to request & sender/receiver users.
Payment	id, requestId, amount, status (PENDING, PAID, FAILED), razorpayOrderId, razorpayPaymentId, signature	1-to-1 payment ledger linked to ServiceRequest with cryptographic validation data.
Review	id, requestId, authorId, subjectId, rating (1-5), comment	Customer feedback linked to request; updates technician average rating dynamically.
ActivityLog	id, userId, action, entity, entityId, details, ipAddress, createdAt	Immutable platform audit log for security telemetry and admin inspection.

3. End-to-End Working Model & Persona Journeys

The platform orchestrates state transitions and real-time events across three core personas:

A. Customer Lifecycle Flow

- **Authentication:** Sign up or instant demo login with dual JWT access/refresh token rotation.
- **5-Step Booking Wizard:** Category selection → Service & base pricing → Symptom checklist & description → Visual attachments (Camera/File/Presets) → Location & scheduling.
- **Smart Dispatch:** Request moves to MATCHING. Server auto-identifies eligible certified technicians in the city and notifies them via WebSockets & RabbitMQ.
- **Live Interaction:** Status updates to ACCEPTED upon technician claim. Customer joins real-time Socket.IO chat room with typing indicators.
- **Execution & Payment:** Technician updates status to IN_PROGRESS → COMPLETED. Customer initiates one-click Razorpay payment (UPI, Card, Netbanking).
- **Review Submission:** Customer submits 1–5 star rating and comment, automatically recalculating technician's public rating metrics.

B. Technician Workspace Flow

- **Onboarding & Verification:** Technician registers trade specialties, years of experience, ID cards, and certifications. Awaits admin approval (PENDING → APPROVED).
- **Availability Management:** Technician toggles availability switch (ONLINE, OFFLINE, BUSY).
- **Job Dispatch Management:** Live incoming request alerts appear on technician dashboard. Technician can inspect diagnostic photos and accept or decline.
- **Job Progress Lifecycle:** Technician transitions request through ACCEPTED → IN_PROGRESS → COMPLETED.
- **Earnings & Performance:** Live dashboard displays total completed jobs, total revenue (INR), and review breakdown.

C. Super Administrator Control Center

- **KPI Metrics:** Platform-wide real-time revenue, booking volume, technician utilization, and completion rates.
- **Technician Verification Queue:** Government ID and certificate inspection with 1-click Approve/Reject actions (triggers automated n8n onboarding email).
- **Global Ticket & Chat Audit:** Inspect any active or historical request across India with full timeline, payment record, and customer-technician chat logs.
- **Security & Activity Logs:** Audit trail capturing user actions, IP addresses, endpoint calls, and timestamps.

4. Asynchronous Task Queues, Kafka & n8n Automation

To ensure zero-blocking HTTP request cycles, asynchronous operations are segregated into message queues and distributed event streams:

Subsystem	Queue / Topic / Blueprint	Function & Error Resilience
RabbitMQ Tasks	fixit.notifications fixit.emails fixit.webhooks fixit.analytics	Dispatches asynchronous in-app notifications, transactional Brevo emails, and telemetry. Failures route to Dead Letter Exchange (fixit.dlx) & fixit.dead_letter.
Kafka Streaming	fixit.request.events fixit.user.events fixit.analytics.events	High-throughput distributed event broker operating in KRaft mode (no Zookeeper). Produces immutable order lifecycle events for telemetry & audit.
n8n Automation	technician-onboarding-alert emergency-escalation payment-receipt-sync low-rating-followup	Visual low-code automation blueprints: Sends onboarding emails, escalates urgent unattended requests, syncs receipts to sheets, and creates support tickets for ratings ≤ 2.

5. Razorpay Payment Gateway & Cryptographic Verification

FixIt embeds a production-grade Razorpay payment integration with server-side price calculations and cryptographic HMAC-SHA256 signature verification:

1. **Order Creation:** Client requests order → Express server fetches base price from DB → Calls razorpay.orders.create({ amount, currency: 'INR' }) → Stores order ID in payments table with status PENDING.
2. **Client Checkout:** Razorpay modal opens with simulated card, UPI (success@razorpay), or Netbanking options in Sandbox mode (or live bank payment in Production).
3. **Signature Verification:** Client posts razorpay_order_id, razorpay_payment_id, and razorpay_signature. Server computes crypto.createHmac('sha256', secret).update(orderId + '|' + paymentId).digest('hex'). If matched, status updates to PAID.

4. **Webhooks Ingestion:** Asynchronous webhook handler processes `payment.captured` and `payment.failed` events to reconcile asynchronous bank settlements.

6. Containerization, Kubernetes & CI/CD Pipeline

The application is engineered for cloud-native deployment with Docker Compose, Kubernetes, and GitHub Actions:

- **Docker Compose:** Orchestrates 7 microservices (postgres, redis, rabbitmq, kafka, n8n, fixit-server, fixit-client) with health check dependencies and profile partitioning.
- **Kubernetes (K8s):** Production manifests in /k8s include StatefulSets for PostgreSQL, Redis, RabbitMQ, and Kafka; Deployments for Server and Client; and Ingress with TLS termination.
- **Horizontal Pod Autoscaler (HPA):** Automatically scales API server pods between 2 and 10 replicas based on real-time CPU (>70%) and Memory (>80%) consumption.
- **GitHub Actions CI/CD:** Automated pipeline runs ESLint, spins up a live PostgreSQL test container for Prisma migrations and API smoke tests, and packages multi-architecture Docker images.

7. Security Hardening & Resilience Engineering

- **Dual-Token JWT Security:** 15-minute access tokens + 7-day rotating refresh tokens stored hashed in DB.
- **Password Security:** 12-round bcrypt hash + single-use 15-minute cryptographically signed reset tokens.
- **Non-Blocking Messaging Resilience:** If Kafka or RabbitMQ is unavailable during cold startup, the REST server logs a warning and proceeds with database operations without dropping user requests.
- **Auto-Seeding Mechanism:** On cold boot, the server checks if database tables are populated; if empty, it automatically runs seeds for categories, services, and default demo accounts.

FixIt Engineering Specification | Designed & Maintained by Aman Singh | Repository: github.com/AmanSingh19/FixIt